

BAB 5

POLYMORPHISM (POLIMORFISME)

5.1 Pendahuluan

Pada BAB 4, mahasiswa telah mempelajari konsep **inheritance**, yaitu bagaimana sebuah class dapat mewarisi atribut dan method dari class lain. Dengan inheritance, kita dapat membuat struktur program yang lebih rapi dan mengurangi duplikasi kode.

Pada BAB 5 ini, mahasiswa akan mempelajari konsep **polymorphism**. Konsep ini memungkinkan sebuah method memiliki banyak bentuk atau perilaku yang berbeda, tergantung pada objek yang menggunakannya.

Polymorphism merupakan salah satu konsep penting dalam pemrograman berorientasi objek. Dengan memahami polymorphism, mahasiswa dapat membuat program yang lebih fleksibel dan mudah dikembangkan.

5.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep polymorphism dalam OOP
2. Memahami perbedaan method overloading dan method overriding
3. Mengimplementasikan method overriding pada subclass
4. Menggunakan polymorphism dalam program Java
5. Memahami manfaat polymorphism dalam pengembangan program

5.3 Pengertian Polymorphism

Polymorphism berasal dari kata:

- **poly** = banyak
- **morph** = bentuk

Sehingga polymorphism berarti **banyak bentuk**.

Dalam OOP, polymorphism adalah kemampuan sebuah method untuk memiliki perilaku yang berbeda tergantung pada objek yang menggunakannya.

Sebagai contoh, kita memiliki method `suara()` pada class `Hewan`. Ketika method ini dipanggil oleh object `Kucing`, maka hasilnya adalah “mengeong”. Ketika dipanggil oleh object `Anjing`, hasilnya menjadi “menggonggong”.

Walaupun nama method sama, hasilnya bisa berbeda. Inilah yang disebut polymorphism.

5.4 Jenis Polymorphism

Dalam Java, polymorphism dibagi menjadi dua jenis utama:

1. Method Overloading
2. Method Overriding

5.5 Method Overloading

Method overloading adalah kondisi di mana **beberapa method memiliki nama yang sama, tetapi parameter berbeda**.

Overloading biasanya digunakan dalam satu class.

Contoh:

```
class Hitung {  
  
    int tambah(int a, int b) {  
        return a + b;  
    }  
  
    int tambah(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Program Utama

```
public class DemoOverloading {  
    public static void main(String[] args) {  
  
        Hitung h = new Hitung();  
  
        System.out.println(h.tambah(5, 3));  
        System.out.println(h.tambah(5, 3, 2));  
    }  
}
```

Output:

8
10

Pada contoh ini, method `tambah()` memiliki dua bentuk berbeda tergantung jumlah parameter.

5.6 Method Overriding

Method overriding adalah kondisi di mana subclass **mengubah atau mengganti method dari superclass**.

Overriding terjadi pada inheritance.

5.6.1 Contoh Dasar Overriding

```
class Hewan {  
    void suara() {  
        System.out.println("Hewan bersuara");  
    }  
}
```

Subclass:

```
class Kucing extends Hewan {  
    void suara() {  
        System.out.println("Kucing mengeong");  
    }  
}  
  
class Anjing extends Hewan {  
    void suara() {  
        System.out.println("Anjing menggonggong");  
    }  
}
```

Output:

Kucing mengeong

Anjing menggonggong

Walaupun variabel bertipe `Hewan`, method yang dipanggil menyesuaikan objeknya. Inilah contoh polymorphism.

5.7 Polymorphism dengan Referensi Superclass

Salah satu kekuatan polymorphism adalah kita bisa menggunakan satu tipe referensi untuk banyak objek.

`Hewan h;`

Variabel `h` bisa digunakan untuk:

- `object Kucing`
- `object Anjing`

Contoh:

```
Hewan h;

h = new Kucing();

h.suara();

h = new Anjing();

h.suara();
```

Ini membuat program lebih fleksibel.

5.8 Contoh Studi Kasus Lengkap

```
class Hewan {
    String nama;

    Hewan(String nama) {
        this.nama = nama;
    }

    void suara() {
        System.out.println("Hewan bersuara");
    }
}
```

```
class Kucing extends Hewan {

    Kucing(String nama) {
        super(nama);
    }

    void suara() {
        System.out.println(nama + " mengeong");
    }
}
```

```
class Anjing extends Hewan {

    Anjing(String nama) {
        super(nama);
    }

    void suara() {
        System.out.println(nama + " menggonggong");
    }
}
```

```

public class DemoPolymorphism {
    public static void main(String[] args) {

        Hewan[] daftarHewan = {
            new Kucing("Milo"),
            new Anjing("Buddy")
        };

        for (Hewan h : daftarHewan) {
            h.suara();
        }
    }
}

```

Output:

Milo mengeong

Buddy menggonggong

5.9 Perbedaan Overloading dan Overriding

Overloading	Overriding
Dalam satu class	Melibatkan inheritance
Parameter berbeda	Nama dan parameter sama
Tidak perlu extends	Harus menggunakan extends
Compile-time	Runtime

5.10 Keuntungan Polymorphism

- Program lebih fleksibel
- Kode lebih rapi dan mudah dikembangkan
- Mendukung reuse code
- Mempermudah penambahan fitur baru

5.11 Kesalahan Umum

- Salah membedakan overloading dan overriding
- Parameter overriding tidak sama
- Tidak menggunakan inheritance
- Tidak memahami konsep referensi superclass

5.12 Tugas Percobaan

Pada bagian ini, mahasiswa diminta untuk mengimplementasikan konsep **polymorphism** secara lebih mendalam melalui studi kasus yang lebih kompleks dan terstruktur.

Percobaan 1: Implementasi Overriding Dasar

Buat class **Hewan** dengan method:

```
void suara()
```

Kemudian buat subclass:

- Kucing
- Anjing
- Burung

Setiap subclass harus **override** method `suara()` dengan perilaku yang berbeda.

Output contoh:

- Kucing → mengeong
- Anjing → menggonggong
- Burung → berkicau

Percobaan 2: Polymorphism dengan Array Object

Buat program yang menyimpan beberapa object dalam sebuah array:

- 2 object Kucing
- 2 object Anjing
- 1 object Burung

Gunakan **array bertipe superclass (Hewan)**, lalu tampilkan suara semua hewan menggunakan perulangan.

Percobaan 3: Overriding + Method Tambahan

Tambahkan method baru pada setiap subclass:

- Kucing → `berburu()`
- Anjing → `bermain()`
- Burung → `terbang()`

Kemudian:

1. Panggil method `suara()` secara polymorphism
2. Panggil method khusus masing-masing object

Percobaan 4: Method Overloading

Buat class `OperasiMatematika` dengan method:

```
tambah(int a, int b)
tambah(int a, int b, int c)
tambah(double a, double b)
```

Tampilkan hasil dari masing-masing method.

Percobaan 5: Studi Kasus Mini (Wajib)

Buat program **Sistem Data Hewan** dengan ketentuan:

Class yang harus dibuat:

- Hewan (superclass)
- Kucing, Anjing, Burung (subclass)

Fitur wajib:

1. Gunakan constructor pada setiap class
2. Gunakan method overriding `suara()`
3. Simpan object dalam array
4. Gunakan perulangan untuk menampilkan data
5. Gunakan minimal 5 object

Output minimal:

- Nama hewan
- Jenis hewan
- Suara hewan

Contoh output:

```
Nama: Milo | Jenis: Kucing | Suara: Mengeong
Nama: Buddy | Jenis: Anjing | Suara: Menggonggong
Nama: Rio | Jenis: Burung | Suara: Berkicau
```

5.13 Tugas Laporan Praktikum

Setelah menyelesaikan seluruh percobaan pada BAB 5, mahasiswa diminta untuk membuat laporan praktikum. Format laporan dibuat konsisten dengan BAB sebelumnya agar mahasiswa terbiasa dengan pola kerja yang sistematis dan rapi.

Struktur laporan:

1. Cover
2. Tujuan praktikum
3. Alat dan bahan
4. Langkah percobaan
5. Source code
6. Hasil output program
7. Analisis program
8. Kesimpulan

Ketentuan isi analisis:

Pada bagian analisis, mahasiswa harus menjelaskan secara rinci implementasi konsep polymorphism pada program yang dibuat. Penjelasan minimal mencakup:

- perbedaan antara **method overloading** dan **method overriding**
- class mana yang menggunakan **overloading**
- class mana yang menggunakan **overriding**
- bagaimana inheritance digunakan untuk mendukung polymorphism
- bagaimana method yang sama dapat menghasilkan output yang berbeda
- penggunaan **referensi superclass** dalam polymorphism
- alur pemanggilan method saat program dijalankan
- perbedaan hasil output dari masing-masing object

Mahasiswa juga diharapkan dapat menjelaskan hubungan antara:

- superclass dan subclass
- method yang diwariskan dan method yang di-override
- serta peran polymorphism dalam membuat program lebih fleksibel

5.14 Penutup

Pada BAB 5 ini, mahasiswa telah mempelajari konsep polymorphism yang memungkinkan method memiliki banyak bentuk. Konsep ini melengkapi inheritance dan menjadikan program lebih fleksibel.

Dengan memahami polymorphism, mahasiswa siap untuk mempelajari konsep lanjutan seperti relasi antar class atau interface pada bab berikutnya.